STUDENT MANAGEMENT SYSTEM

SYSTEM DOCUMENTATION

SUBMITTED BY,
EMMANUEL KIBET
DIT-01-0053/2025

A SYSTEM DOCUMENTATION SUBMITTED IN PARTIAL FULFILMENT FOR THE AWARD OF DIPLOMA IN INFORMATION TECHNOLOGY BY ZETECH UNIVERSITY

APRIL 2026

DECLARATION

I, Emmanuel Kibet, hereby declare that this project titled Student Management System is my original work and has been carried out by me under the guidance of my lecturer and supervisor. I confirm that all sources used in this project have been properly acknowledged and cited where necessary.

I further declare that this project has not been submitted, either in part or in full, for any other academic award or qualification at this or any other institution.

Student Name: _____
Sign: _____
Date: _____

DEDICATION

I dedicate this project to my parents, for their endless support, guidance and encouragement throughout my studies.

I also dedicate this work to my lecturers and classmates, who have contributed to my learning journey through their guidance, advice and motivation.

Finally, I dedicate this project to anyone who aspires to develop innovative systems and solutions as a small contribution to the field of technology.

ABSTRACT

This project presents a Student Management System is a web-based application designed to simplify the management of student information. The system allows students and administrators to perform tasks such as registration, course management, fee tracking and dashboard access based on user roles.

The system was developed using PHP for server-side logic, MySQL for the database and HTML/CSS for the interface. Validation checks ensure secure login, proper registration and controlled access to dashboards.

Testing was conducted to ensure all features function correctly and corrections were made to improve reliability. The system was deployed using XAMPP allowing access through a web browser. The project provides an efficient, secure and user-friendly platform for managing student data.

```
<div id="unitModal" class="modal">
<div class="modal-content">
<div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid #eee; margin-bottom:20px;">
<h3 style="margin:0;">Add New Curriculum Unit</h3>
<span onclick="document.getElementById('unitModal').style.display='none'" style="cursor:pointer; font-size:24px;">&times;</span>
</div>

<form method="POST" action="admin.php?tab=units">
<label>Unit Name</label>
<input type="text" name="unit_name" placeholder="e.g. Database Management" required>

<label>Unit Code</label>
<input type="text" name="unit_code" placeholder="e.g. CIT 2104" required>

<div class="form-grid">
<div>
<label>Year</label>
<select name="year" required>
<option value="1">Year 1</option>
<option value="2">Year 2</option>
</select>
</div>
<div>
<label>Semester</label>
<select name="semester" required>
<option value="1">Semester 1</option>
<option value="2">Semester 2</option>
</select>
</div>
</div>

<label>Assign to Course</label>
<select name="course_id" required>
<?php
$courses = $conn->query("SELECT * FROM courses");
while($c = $courses->fetch_assoc()) {
echo "<option value='".$c['id']."'>".$c['course_name']."</option>";
}
?>
</select>
```

```
<button type="submit" name="save_unit" class="bg-green" style="width:100%; color:white;
border:none; padding:12px; margin-top:15px; cursor:pointer; font-weight:bold; border-radius:3px;">
SAVE UNIT
</button>
</form>
</div>
```

** DEFINITION OF KEY TERMS**

1. Student Management System (SMS):
A software application designed to automate and manage student-related information such as registration, courses, fee payments, and academic records.

2. Dashboard:
An interface that provides users with a summary of their data and quick access to system features based on their role, e.g., student or administrator.

3. User Role:
A designation assigned to a system user that determines the level of access they have, such as **Admin** or **Student**.

4. Database:
An organized collection of data stored electronically, which allows the system to store, retrieve, update, and delete information efficiently.

5. Validation:
A process that ensures the data entered into the system is correct, complete, and meets the expected rules before being processed or stored.

6. Deployment:
The process of making the system available for use by users, typically by installing it on a server where it can be accessed through a web browser.

7. Session:
A way to store temporary information about a user while they navigate the system, commonly used to track login status and user role.

8. Login Authentication:
The process of verifying a user's identity by checking the credentials (username and password) against the database records.

ABBREVIATIONS AND ACRONYMS

- **HTML –** Hyper Text Mark Up Language
- **SQL –** Structured Query Language
- **PHP** – Hypertext Preprocessor
- **CSS** – Cascading Style Sheets
- **IDE** – Integrated Development Environment
- **DBMS** – Database Management System
- **VS Code** – Visual Studio Code
- **UI** – User Interface

LIST OF FIGURES

Fig 2.2.1 Login Page Design
Fig 2.2.2 Sign Up Form Design
Fig 2.2.3 Dashboard Form Design
Fig 2.2.4 Records Management Page Design
Fig 3.2.1 Login Page Interface
Fig 3.2.2 Forgot Password Page
Fig 3.2.3 Sign Up Page

Fig 3.2.4 Admin Dashboard
Fig 3.2.5 Student Dashboard

LIST OF TABLES

Table 1.2 Functional Requirements
Table 1.3 Budget Breakdown
Table 1.4: Functional Requirements Table
Table 1.6**:** Project Schedule Breakdown
Table 3.3.6.1: Users Table (Database Schema)
Table 3.3.6.2: Students Table (Database Schema)
Table 3.3.6.3: Courses Table (Database Schema)
Table 3.3.6.4: Fees Table (Database Schema)

TABLE OF CONTENTS

Contents

DECLARATION 1
DEDICATION 3
ABSTRACT 4
DEFINITION OF KEY TERMS 5
ABBREVIATIONS AND ACRONYMS i
LIST OF FIGURES i
TABLE OF CONTENTS iii
CHAPTER ONE: PROJECT PLANNING AND ANALYSIS 1
(WORKPLAN) 1
1.1 Statement of Problem 1
1.2 Study Justification 1
1.3 System Objectives 1
1.3.1 General Objective 1
1.3.2 Specific Objectives 1

1.4 Functional Requirements 1
1.5 Breakdown of Tools & Resources to Be Used 1
1.6 Project Schedule Breakdown 1
CHAPTER TWO: DESIGN AND MODELING 2
2.1 INTRODUCTION TO MODELLING 2
2.2 USER INTERFACE MODEL 2
2.2.1 LOGIN PAGE 2
2.2.2 SIGN UP FORM DESIGN 2
2.2.3 DASHBOARD FORM DESIGN 3
2.3 Logic Models 5
2.3.1 Login Page Flowchart 5
2.3.1 Sign Up Page Flowchart 6
2.3.3 DASHBOARD PAGE FLOWCHART 7
CHAPTER THREE: SYSTEM IMPLEMENTATION 10
(DEVELOPMENT, TESTING AND DEPLOYMENT) 10
3.1 Introduction 10
3.2 User Interface Development 10
3.2.1 Login Development 10
3.2.2 Forgot Password Development 13
3.2.3 Sing Up Development 14
3.2.4 Admin Dashboard Development 18
3.2.5 Student Dashboard Development 19
3.3 Logic Development 20
3.3.1 login validation code 20
3.3.2 Forgot password validation code 21
3.3.3 Sign Up Development Validation Code 21
3.3.4 Admin Dashboard Validation Code 22
3.3.5 Student Dashboard Validation Code 22
3.3.6 DATABASE DESIGN 23
3.4 Testing 24
3.5 Deployment 25
CHAPTER FOUR: CONCLUSION AND RECOMMENDATION 25
4.1 Conclusion 25
4.2 Recommendation 25
REFERENCES 26

CHAPTER ONE: PROJECT PLANNING AND ANALYSIS

(WORKPLAN)

1.1 Statement of Problem

The current way that academic information is shared in the university is through manual work. Students have to physically go to an information desk, where they are attended to by a staff member who checks paper files, books, or individual computer screens to provide information regarding schedules and examinations. This manual approach presents several major problems. First, it results in long wait times because as the number of students increases, the staff are unable to cope, leading to long queues. Second, it creates academic risk since the information given to students is often not timely, causing some to miss important academic deadlines. Third, the process leads to high stress levels for both students and staff, as staff members must repeatedly provide the same information manually. Finally, the system is inefficient; while other universities are adopting modern methods of information sharing, this university continues to lag behind with slow and inefficient practices.

1.2 Study Justification

This study is necessary to find a better way to provide information to the students. Moving from a manual system to a technological solution is important for several reasons. First, it will improve speed and access, as technology would enable students to obtain the required information in a timely manner without the need to wait in lines. Second, it will enhance accuracy by ensuring that students do not miss important deadlines due to communication delays. Third, it will create an improved environment by reducing the stress levels of both university staff and the student body, since the need to perform repetitive actions manually will be minimized. Finally, modernization is essential for the university to align its services with global standards and to effectively accommodate the growing student population.

1.3 System Objectives

1.3.1 General Objective

To design and develop an automated student information system that enhances the efficient, accurate and timely management of academic information within the university.

1.3.2 Specific Objectives

(i) To design a user-friendly automated student information system for students.
(ii) To implement secure login and access control for different types of users the admin and students.
(iii) To develop a student management module for capturing, updating, and storing student records.
(iv) To design and implement a user-friendly dashboard interface for both admin and students.

1.4 Functional Requirements

User	**User Activities**	**Features**
Admin	- update student information - upload academic dates and schedules	- Data Management. - Admin Dashboard.
Student	- Log in and search for schedules, grades, course, fee and exam dates	- Search Bar Personal Profile

Table 1.4 Functional Requirements Table

1.5 Breakdown of Tools & Resources to Be Used

1. Software Tools

- Front-End: HTML, CSS, JavaScript – for building the user interface, styling the pages, and making the search bars interactive.
- Back-End: PHP or Python (Django/Flask) – to handle the system logic, user logins, and information retrieval.
- Database: MySQL or SQLite – to securely store and organize student records, exam dates, and class schedules.
- Security: SHA-256 Hashing – for protecting student passwords and ensuring data privacy.

2. Development Environment

- IDE: Visual Studio Code (VS Code) – for writing and debugging the project code.
- Local Server: XAMPP or WAMP – to run the database and the system locally on a computer during development.
- Version Control: Git & GitHub – for saving different versions of the code and tracking progress.

3. Deployment Resources

- 000webhost or Render: Free hosting services to make the system accessible online for testing.
- GitHub Pages: To host the project documentation and the static front-end design.
- Localhost: For demonstrating the system's functionality during the final presentation.

4. Hardware Resources

- Development Laptop: A computer with at least 4GB RAM to build and test the software.
- Client Devices: Smartphones and tablets to ensure the system is responsive and easy for students to use on mobile.
- Internet Router: To provide the connectivity needed for research and cloud database synchronization.

5. Human Resources

- System Developer: You, the student responsible for designing, coding, and testing the system.
- System Supervisor: The lecturer who provides professional guidance and reviews the project milestones.
- Pilot Users: A small group of fellow students and staff who will test the system to find any bugs or errors.

1.6 Project Schedule Breakdown

	PROJECT MILESTONES					
WEEKS	**Project Planning & Analysis** **(System Documentation: Coverpage & Chapter One)**	**Project Design & Modeling** **(System Documentation Chapter Two)**	**Project Development & Testing** **(System Documentation Chapter Three)**	**Project Deployment** **(System Documentation Chapter Three)**	**Final Touches of System Documentation** **(Preliminary Pages, Chapter Four & References)**	**Project Presentation**
5-9 May						
12-16 May						
19-23 May						
26-30 May						
2-6 June						
9-13 June						
16-20 June						
23-27 June						
30-4 July						
7-11 July						
14-18 July						
21-25 July						
11 Aug						

Table 1.6 Project Schedule Breakdown

Blue initially means that milestone has not achieved. Once you achieve that milestone, fill it with any other color of your choice to mean you're done with that part. That helps you to monitor your own progress versus the time scheduled.

When printing, this page must be printed in LANDSCAPE as the rest is printed in portrait

CHAPTER TWO: DESIGN AND MODELING

2.1 INTRODUCTION TO MODELLING

This chapter presents the design and modelling techniques that will be used to visualize the proposed system before the actual development work begin. System modelling is an important step in this project because it will help to translate system requirements into clear visual representations that could be easily understood by both technical and non-technical stakeholders. By developing models this user interface sketches, flowcharts and structural diagrams, it will possibly identify potential design issues early, improve system usability, and ensure that all functional requirements are fully addressed before implementation of the system. Overall, modelling reduced development risks, saved time and provided a clear overview for building the system.

2.2 USER INTERFACE MODEL

This section presents the user interface (UI) models that are going to be used to design and illustrate how different users will interact with the system. The interfaces are initially hand sketched and later refined using shapes in a Microsoft Word document. These models guide the development of a user-friendly, consistent and intuitive system interfaces.

2.2.1 LOGIN PAGE

The login page design is created to allow registered users to securely access the system. The interface includes input fields where the user can enter either the username or registration number together with a password and then click the login button.. The simplicity of the design ensures ease of use and quick access to the system.

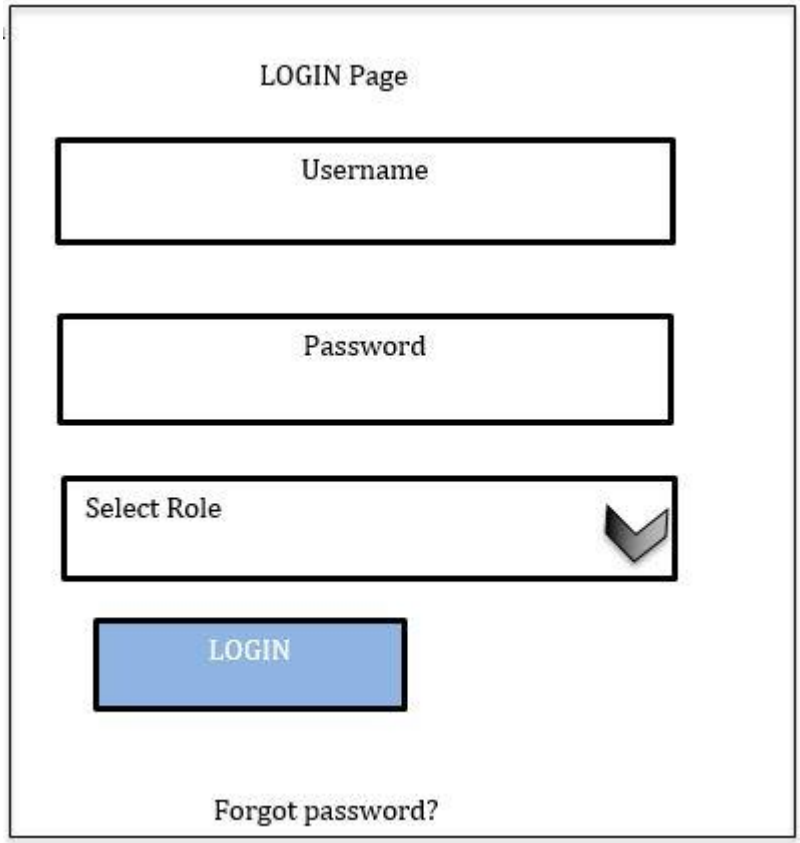


Fig 2.2.1 Login Page Design

2.2.2 SIGN UP FORM DESIGN

The sign-up form design allows admin to add new users to the system. It captures important user details such as registration number, personal information, course details and the login credentials. When the admin enters the details the student is automatically added to the system. Validation is considered at the design stage to ensure data accuracy.

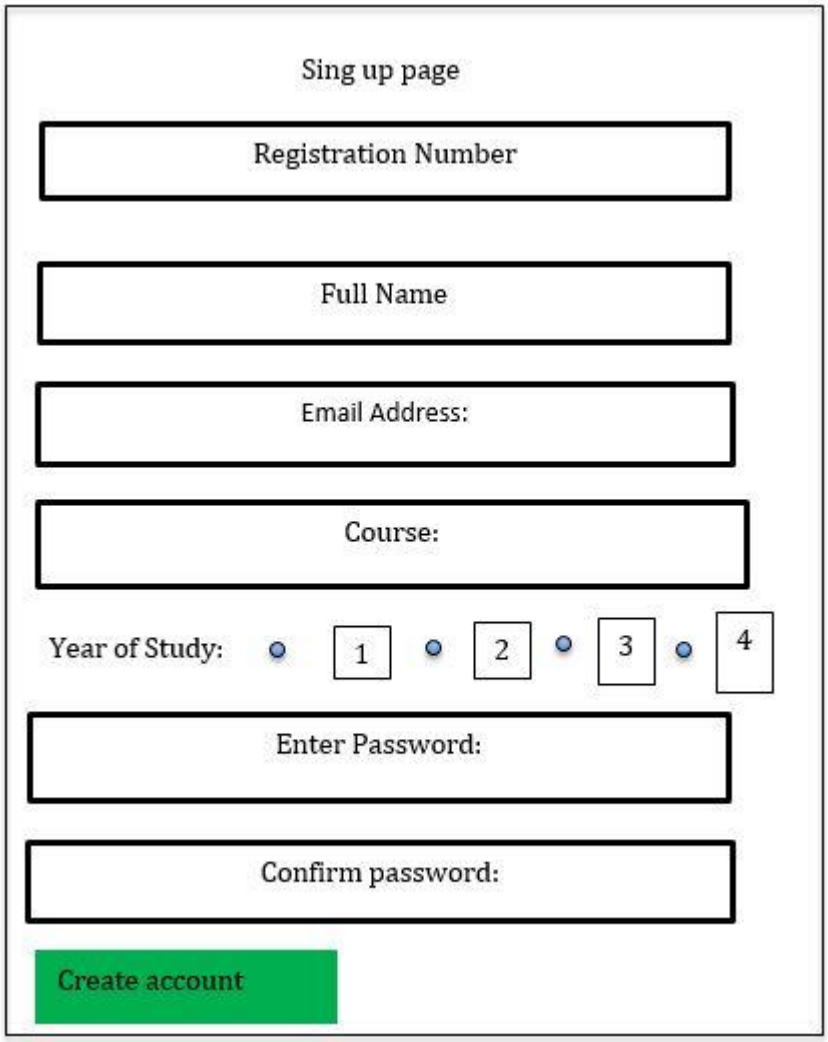


Fig 2.2.2 Sign-Up Form Design

2.2.3 DASHBOARD FORM DESIGN

The dashboard form design provides users with an overview of important system information after successful login. It displays key academic details such as notifications, announcements and quick access menus to major system functions. The design

improves system usability by enabling users access their frequently used features smoothly from a single screen.

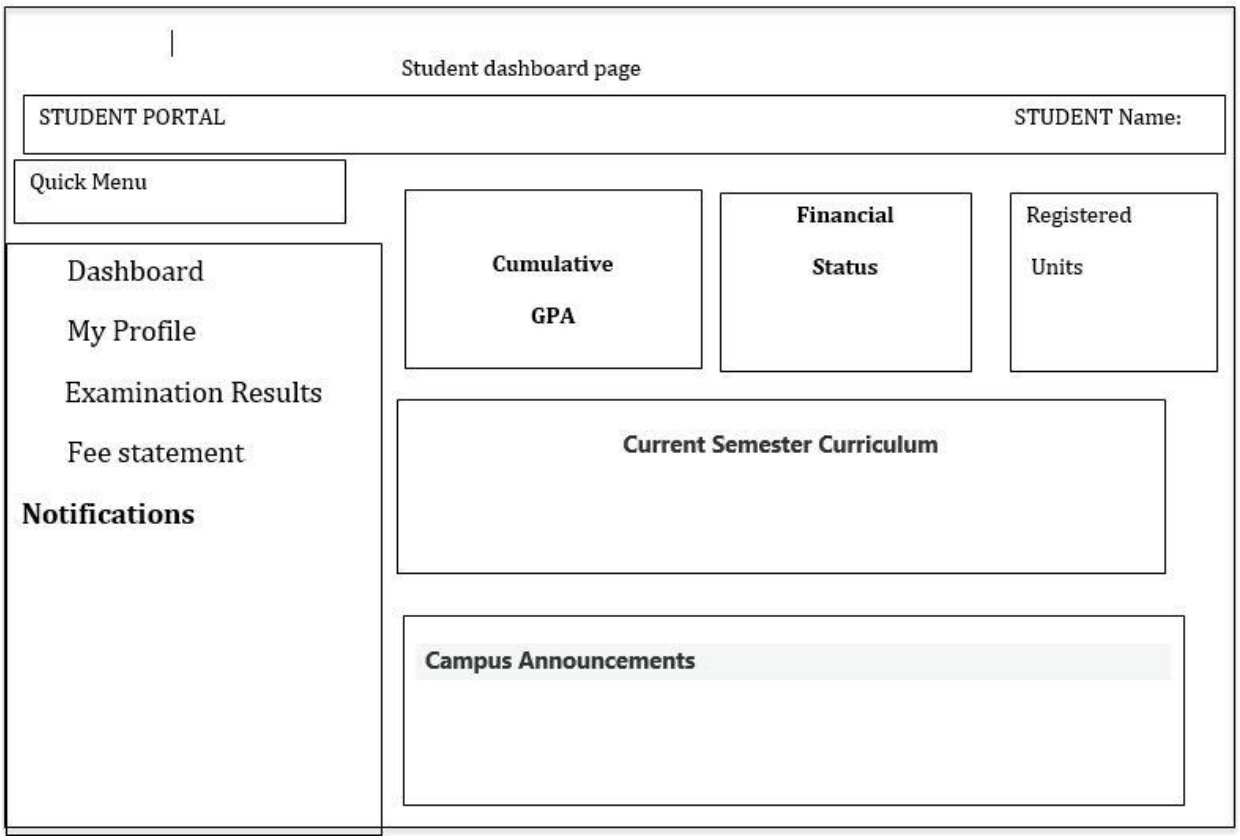


Fig 2.2.3 Dashboard Form Design

2.2.4 RECORDS/MANAGEMENT PAGE DESIGN

The records management page design allows administrators to manage student information within the system. It also provides tools for adding, viewing, updating and deleting student records. The table display ensures that the records are displayed clearly and neat, while action buttons enable productive record management. This design supports accuracy, security and easy of

administration to manage the records.

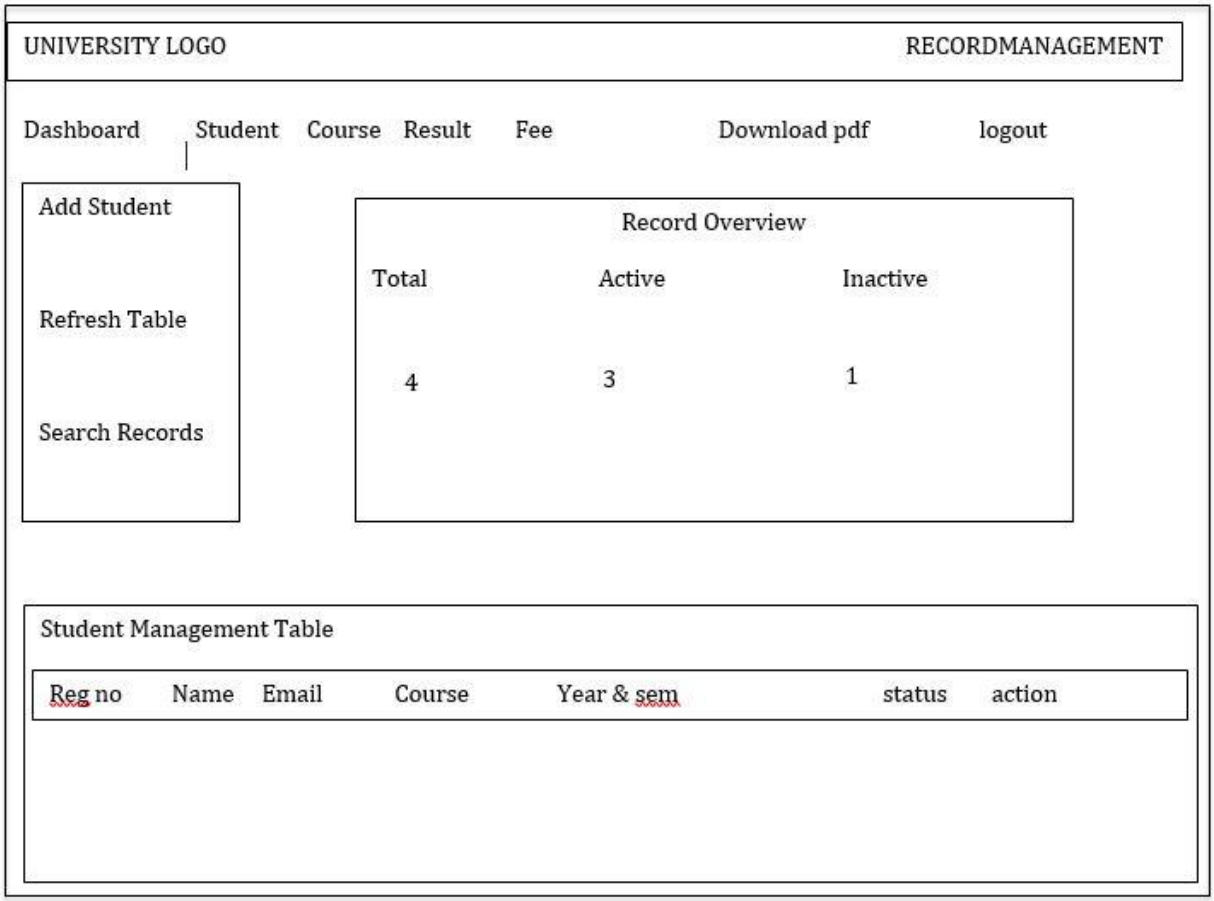
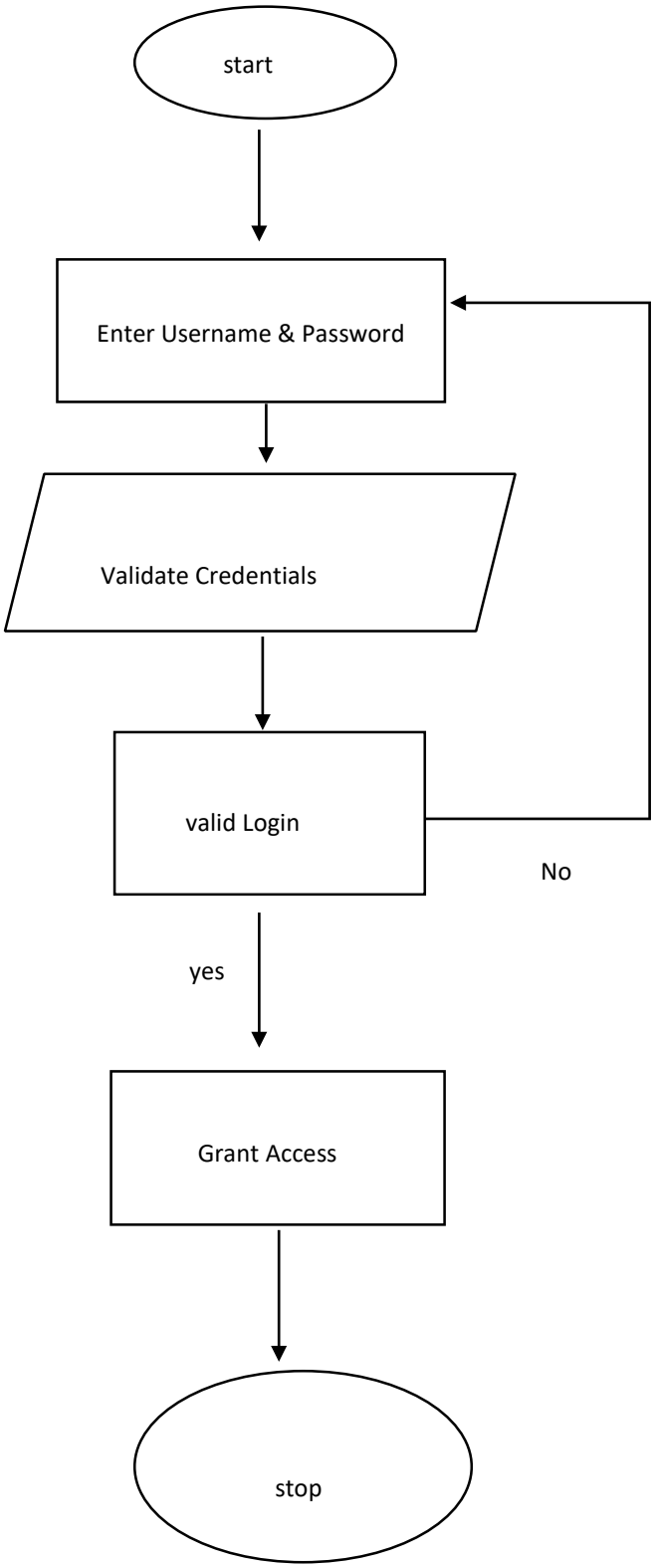


Fig 2.2.4 RECORDS/MANAGEMENT PAGE DESIGN

2.3 Logic Models

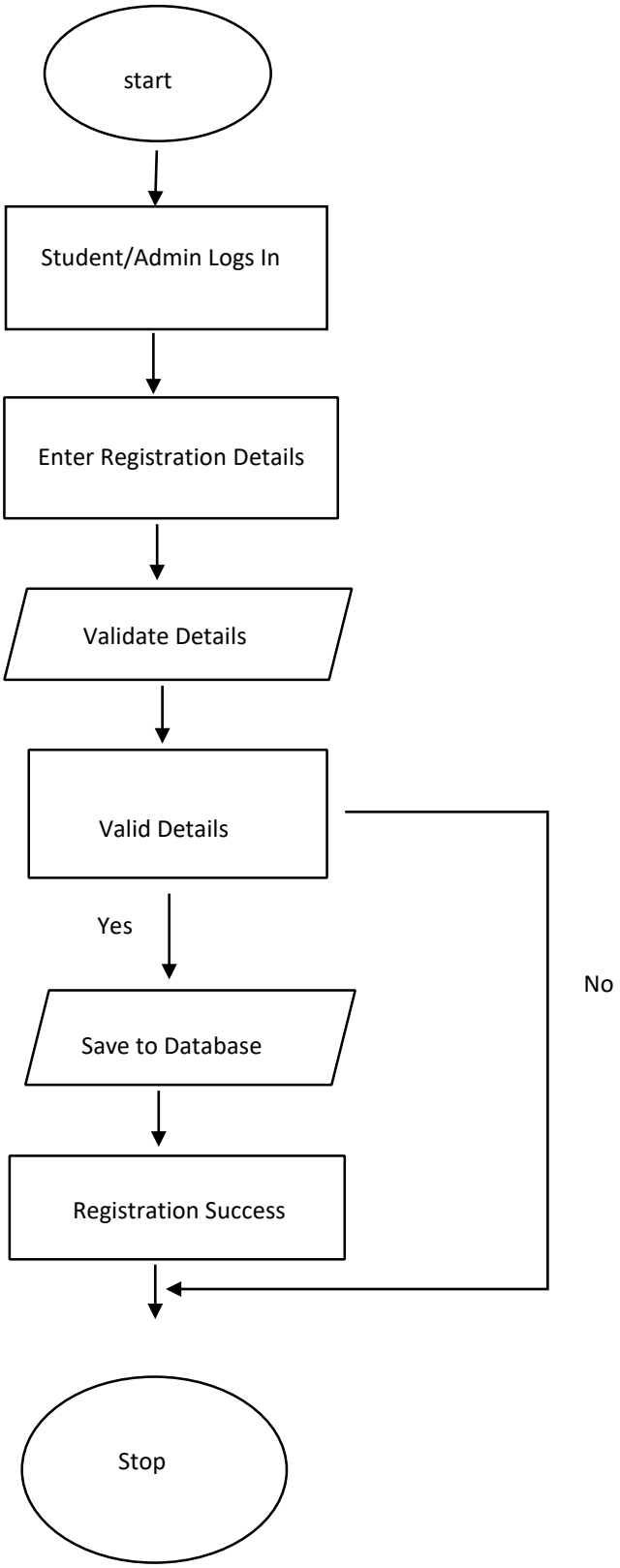
2.3.1 Login Page Flowchart

The log in page flow chart is used to show how data is flowing step by step when the user try to access the system. It starts will entering the credentials where if the credentials are valid it grants access to the user but if it is invalid it denies the access.



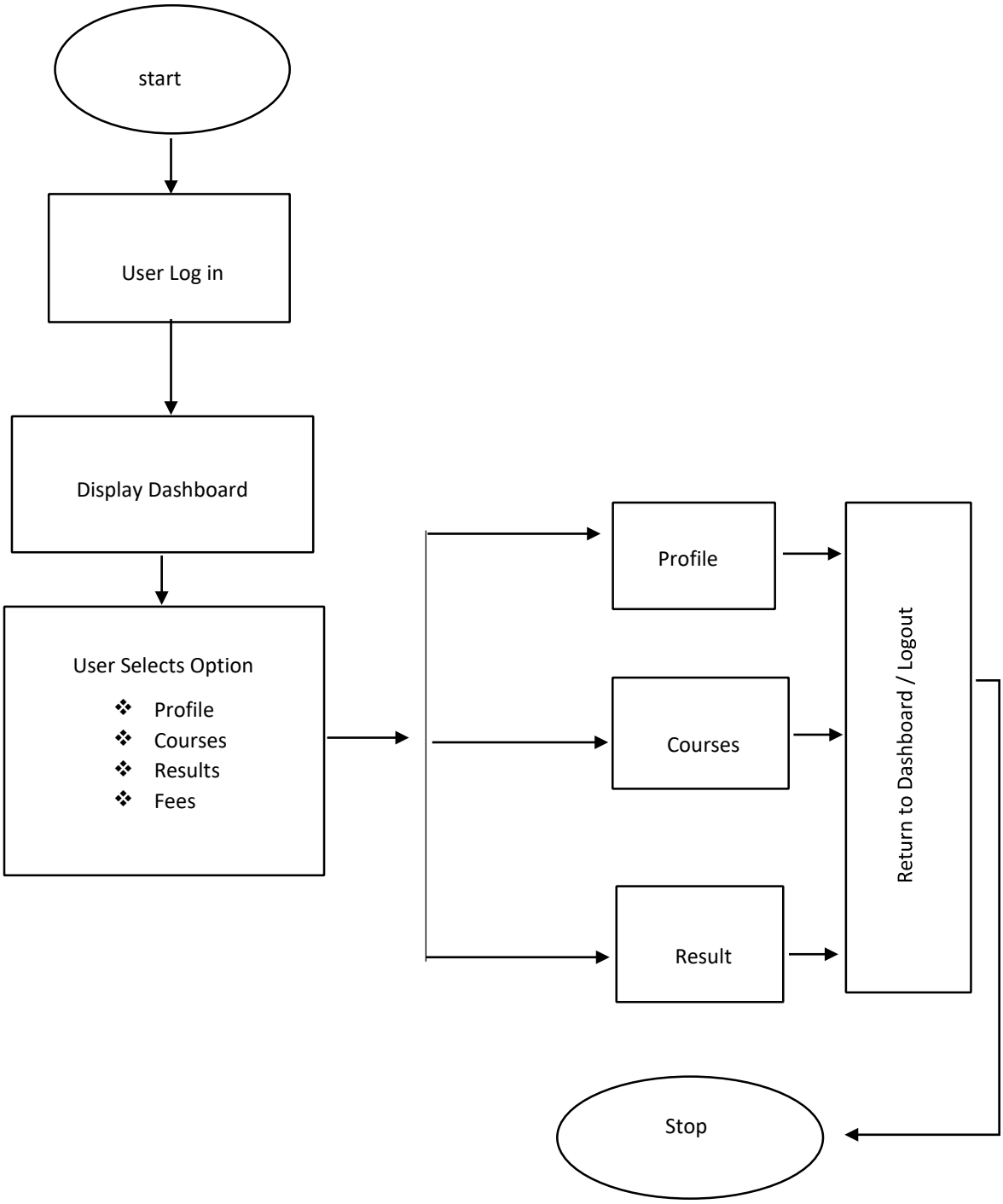
2.3.1 Sign Up Page Flowchart

Sing up page flowchart shows how data flow when the user visits the system for the first time and needs to register themselves to the system. The user keys in the details and if the details are valid it is saved to the admin database but if invalid it denies registration



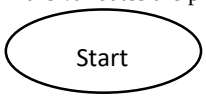
2.3.3 DASHBOARD PAGE FLOWCHART

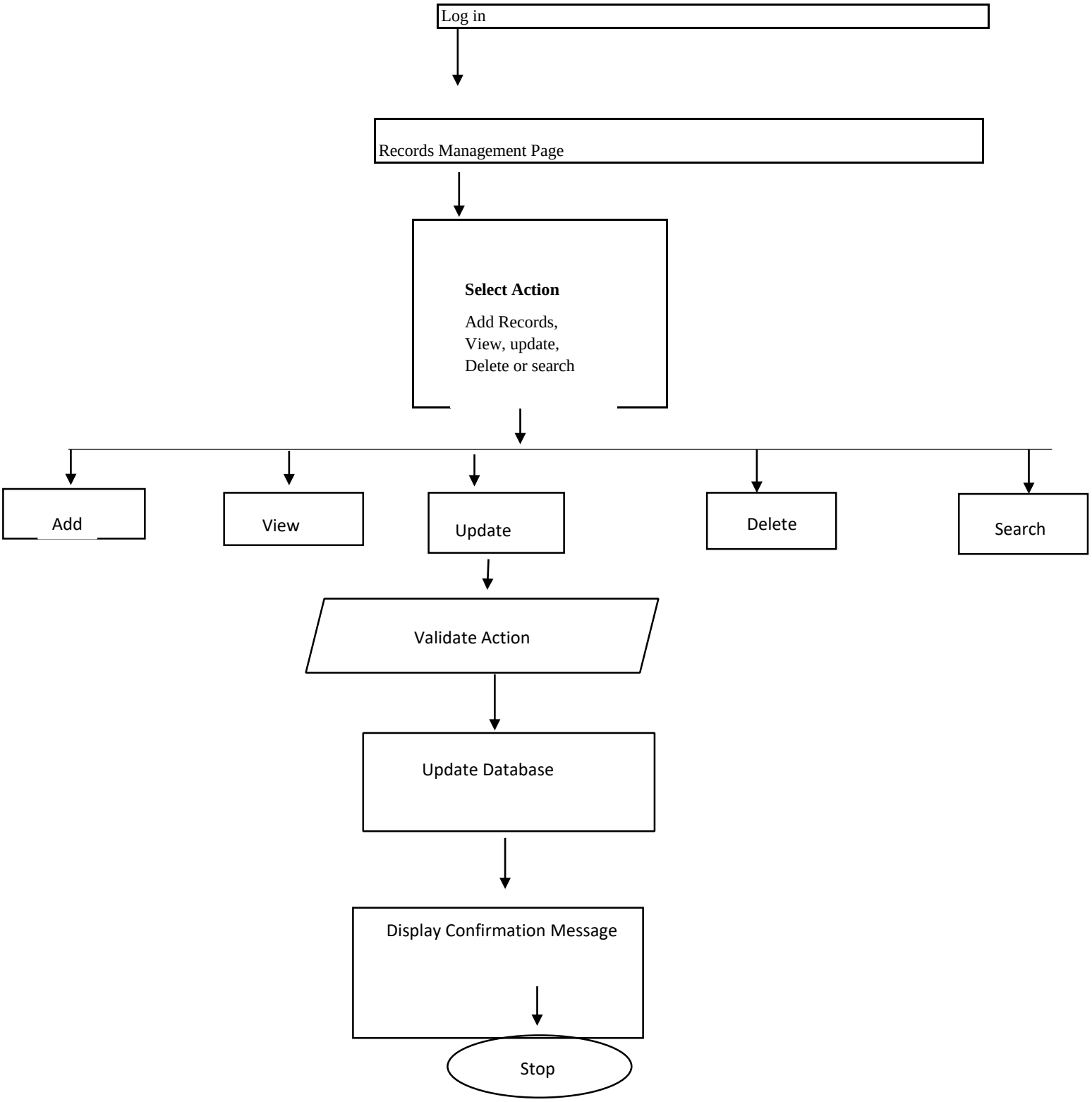
This shows how the overview of the system looks like the user log in to the system. It also gives the user different options of where they want to visit to the system directly at the dashboard this making it easy for the user to use and easy to understand the system.



2.3.4 RECORD/MANAGEMENT PAGE FLOWCHART

This flowchart shows how administrator manages student records within the system. The administrators can choose to add, view, update delete or search records. The system the validates the process and the grant access to the admin




Log in
Records Management Page
Select Action
Add Records,
View, update,
Delete or search
Add
View
Update
Delete
Search
Validate Action
Update Database
Display Confirmation Message
Stop

CHAPTER THREE: SYSTEM IMPLEMENTATION

(DEVELOPMENT, TESTING AND DEPLOYMENT)

3.1 Introduction

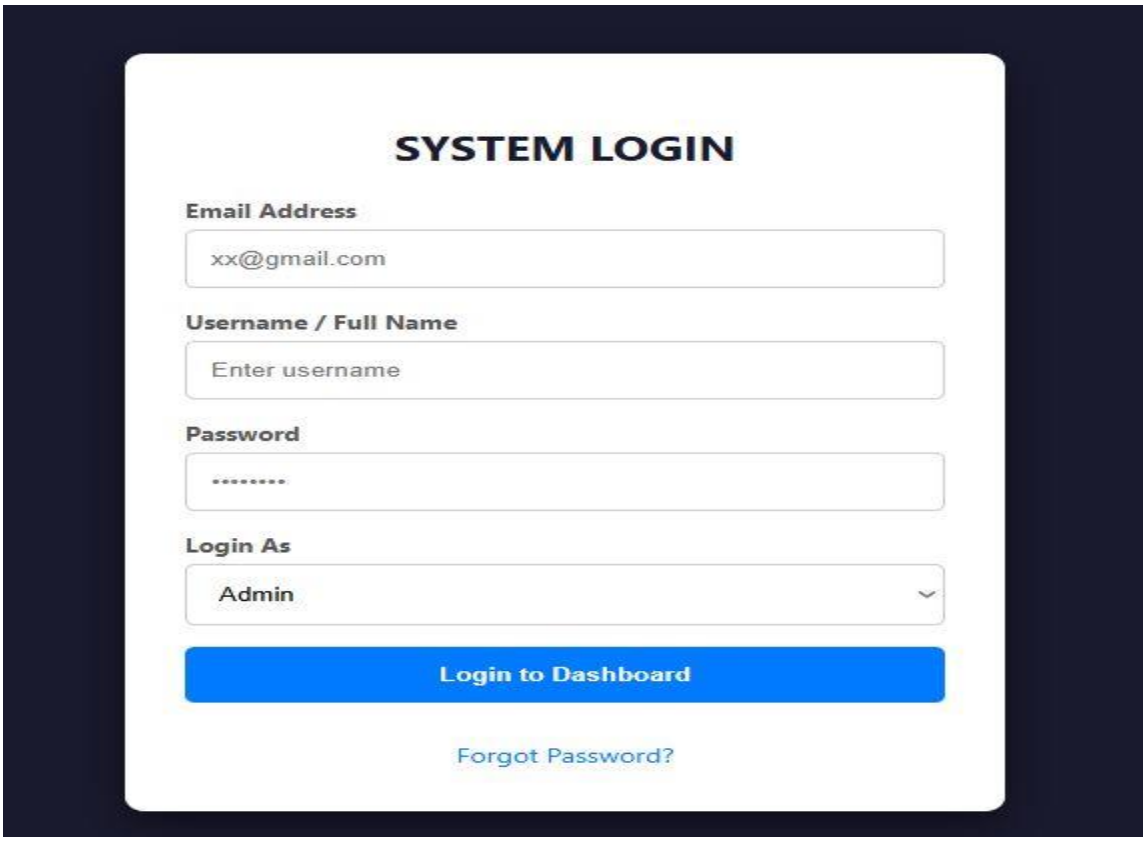
In this chapter I describe the process I followed to develop, test, and deploy the Student Management System**.** I took several steps from setting up the development environment to building the different components of the system. During the development process, I installed Visual Studio Code which I used in my front end development which holds the structure of the website which displays the details and also XAMPP. These are the software that I used in my web development to design and implement the system interface and functionality.

I used XAMPP to create and implement the database which will be used to store and manage student information such as registration details, login credentials, and other records. After developing the system, I carried out testing to ensure that the different features work correctly and that the system performs as expected. Finally, I deployed the system so that it can be accessed and used by users who are admins and the student. This chapter therefore presents the practical steps and tools I used in implementing the Student Management System.

3.2 User Interface Development

3.2.1 Login Development

This is the page where the users will enter their credentials where they will be verified before being granted access to their respective dashboards according to their role. If the user is a student, they are directed to the student dashboard, while the admin is directed to the admin dashboard.


SYSTEM LOGIN
Email Address
xx@gmail.com
Username / Full Name
Enter username
Password
Login As
Admin
Login to Dashboard
Forgot Password?

Login Page

```html
<head>
    <style>
    </style>
</head>
<body>
    <div class="card">
        <h2>SYSTEM LOGIN</h2>
        <?php if($error_msg) echo "<div class='error'>$error_msg</div>"; ?>
        <form method="POST">
            <label>Email Address</label>
            <input type="email" name="email" placeholder="xx@gmail.com" required>

            <label>Username / Full Name</label>
            <input type="text" name="username" placeholder="Enter username" required

            <label>Password</label>
            <input type="password" name="password" placeholder="••••••••" required>

            <label>Login As</label>
            <select name="role">
                <option value="Admin">Admin</option>
                <option value="Student">Student</option>
            </select>

            <button type="submit" name="login_btn">Login to Dashboard</button>
        </form>
        <div class="footer-links">
            <a href="forgot_password.php">Forgot Password?</a>
        </div>
    </div>
</body>
```

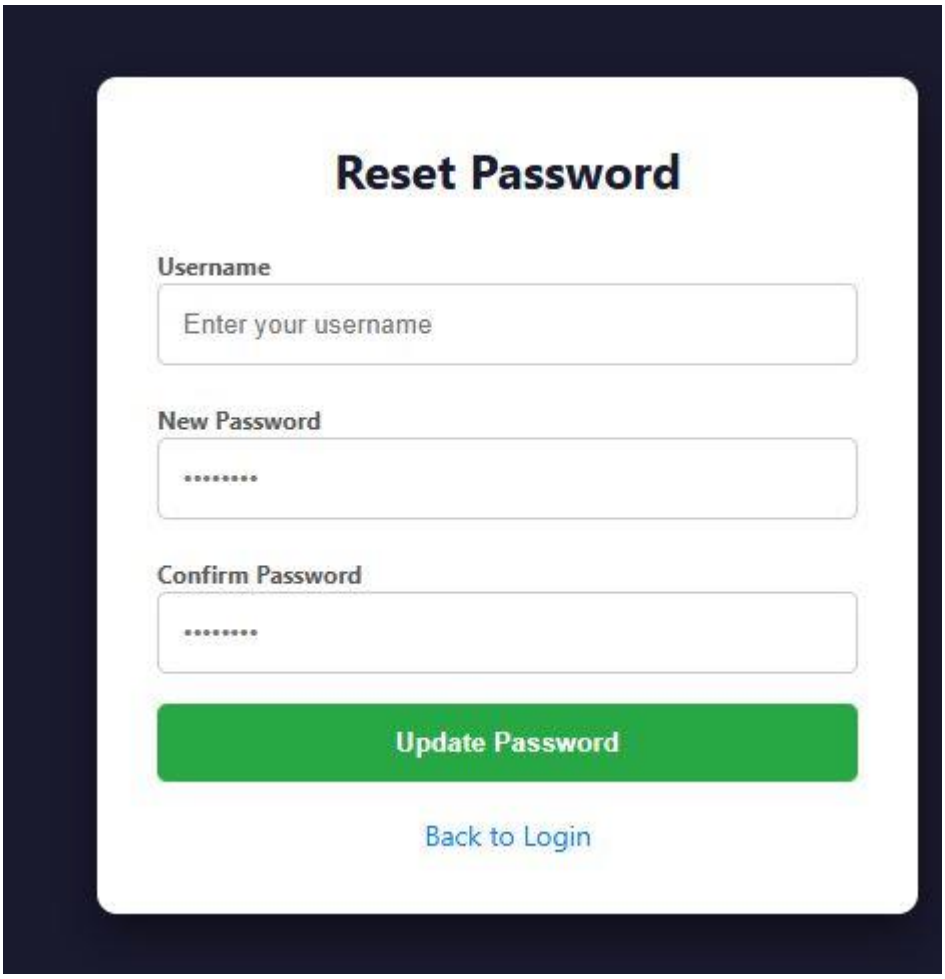
Screenshot of its html

```style
<style>
    body { background: #1a1a2e; font-family: 'Segoe UI', sans-serif;
    display: flex; justify-content: center; align-items: center; min-height: 100vh; margin: 0; }
    .card { background: white; padding: 30px; border-radius: 10px; width: 380px; box-shadow: 0 10px 25px rgba(0,0,0,0.5); }
    h2 { text-align: center; color: #1a1a2e; margin-bottom: 20px; }
    label { font-size: 13px; font-weight: bold; color: #555; display: block; margin-bottom: 5px; }
    input, select, button { width: 100%; padding: 12px; margin-bottom: 15px;
     border-radius: 5px; border: 1px solid #ccc; box-sizing: border-box; }
    button { background: #007bff; color: white; border: none; cursor: pointer; font-weight: bold; transition: 0.3s; }
    button:hover { background: #0056b3; }
    .error { background: #f8d7da; color: #721c24; padding: 10px; border-radius: 5px;
     margin-bottom: 15px; text-align: center; font-size: 14px; }
    .footer-links { text-align: center; margin-top: 10px; }
    .footer-links a { color: #007bff; text-decoration: none; font-size: 14px; }
    .footer-links a:hover { text-decoration: underline; }
</style>
```

Screenshot of its css

3.2.2 Forgot Password Development

This is where the user interacts with when they forget their password and they need to access their portal so they request to set a new password where it is set and saved.



Forgot Password Page

```html
</head>
<body>

<div class="login-container">
    <h2>Reset Password</h2>

    <?php if($msg): ?>
        <div class="msg <?php echo $msg_class; ?>"><?php echo $msg; ?></div>
    <?php endif; ?>

    <form method="POST" action="">
        <label style="font-size: 12px; font-weight: bold; color: #555;">Username</label>
        <input type="text" name="usernameInput" placeholder="Enter your username" required>

        <label style="font-size: 12px; font-weight: bold; color: #555;">New Password</label>
        <input type="password" name="newPassword" placeholder="••••••••" required>

        <label style="font-size: 12px; font-weight: bold; color: #555;">Confirm Password</label>
        <input type="password" name="confirmPassword" placeholder="••••••••" required>

        <button type="submit">Update Password</button>

        <div class="back-link">
            <a href="login.php">Back to Login</a>
        </div>
    </form>
</div>

</body>
</html>
```

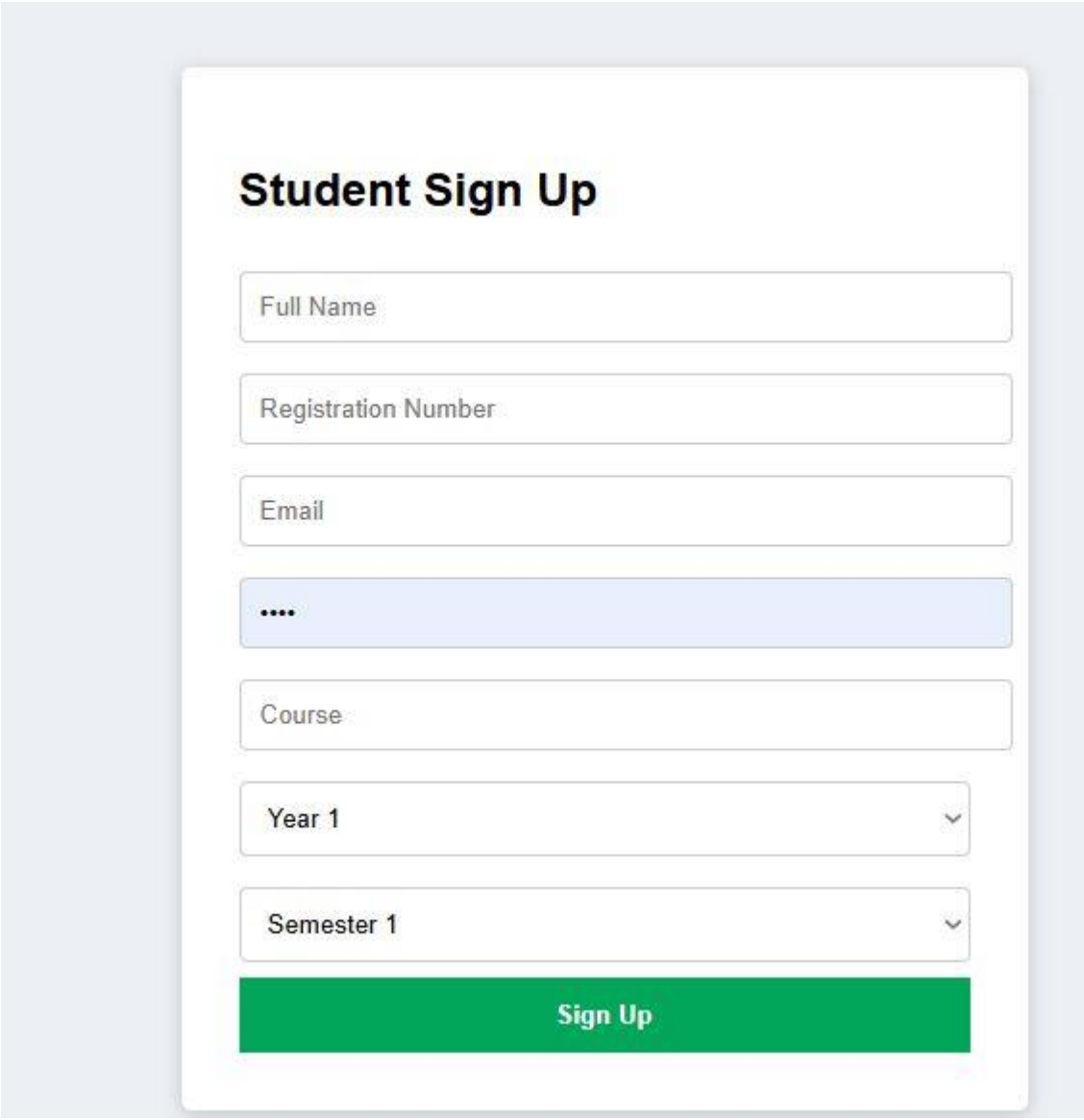
Screenshot of its html

```style
<style>
    body { background: #1a1a2e; font-family: 'Segoe UI', sans-serif;
    display: flex; justify-content: center; align-items: center; height: 100vh; margin: 0; }
    .login-container { background: white; padding: 30px; border-radius: 10px; width: 350px; box-shadow: 0 10px 25px rgba(0,0,0,0.5); }
    h2 { text-align: center; color: #1a1a2e; margin-top: 0; }
    input { width: 100%; padding: 12px; margin-bottom: 15px; border-radius: 5px; border: 1px solid #ccc; box-sizing: border-box; }
    button { width: 100%; background: #28a745; color: white; padding: 12px;
    border: none; border-radius: 5px; cursor: pointer; font-weight: bold; }
    button:hover { background: #218838; }
    .msg { padding: 10px; margin-bottom: 15px; border-radius: 5px; text-align: center; font-size: 14px; }
    .error { background: #f8d7da; color: #721c24; border: 1px solid #f5c6cb; }
    .success { background: #d4edda; color: #155724; border: 1px solid #c3e6cb; }
    .back-link { text-align: center; margin-top: 15px; }
    .back-link a { color: #007bff; text-decoration: none; font-size: 14px; }
</style>
```

Screenshot of its css

3.2.3 Sing Up Development

This is where the management enters details of a new student where the data goes to database and when the admin opens the portal displays the student added to student list.



Register Page

```html
<div class="box">

<h2>Student Sign Up</h2>

<?php if($message!=""){ echo "<p>$message</p>"; } ?>

<form method="POST">

<input type="text" name="full_name" placeholder="Full Name" required>

<input type="text" name="reg_number" placeholder="Registration Number" required>

<input type="email" name="email" placeholder="Email" required>

<input type="password" name="password" placeholder="Password" required>

<input type="text" name="course" placeholder="Course" required>

<select name="year">
<option value="1">Year 1</option>
<option value="2">Year 2</option>
</select>

<select name="semester">
<option value="1">Semester 1</option>
<option value="2">Semester 2</option>
</select>

<button type="submit" name="signup">Sign Up</button>
```

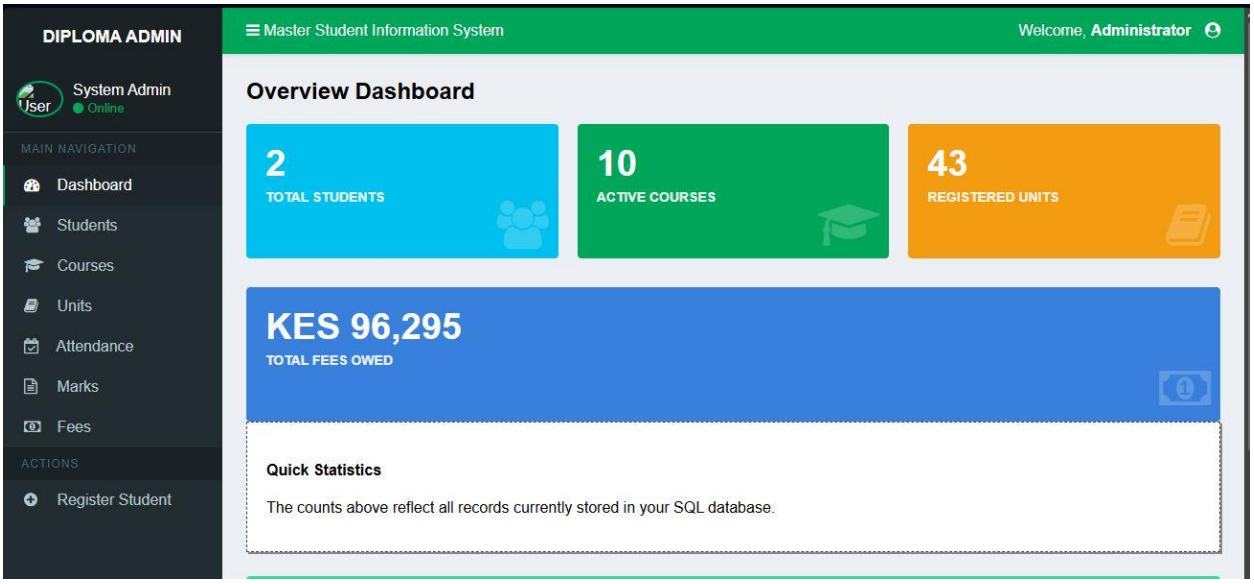
Screenshot of it html

```
<style>

body{
font-family:Arial;
background:#ecf0f5;
display:flex;
justify-content:center;
align-items:center;
height:100vh;
}

.box{
background:white;
padding:30px;
width:380px;
border-radius:5px;
box-shadow:0 2px 10px rgba(0,0,0,0.2);
}

input,select{
width:100%;
padding:10px;
margin:8px 0;
border:1px solid #ccc;
border-radius:4px;
}

button{
width:100%;
padding:12px;
```

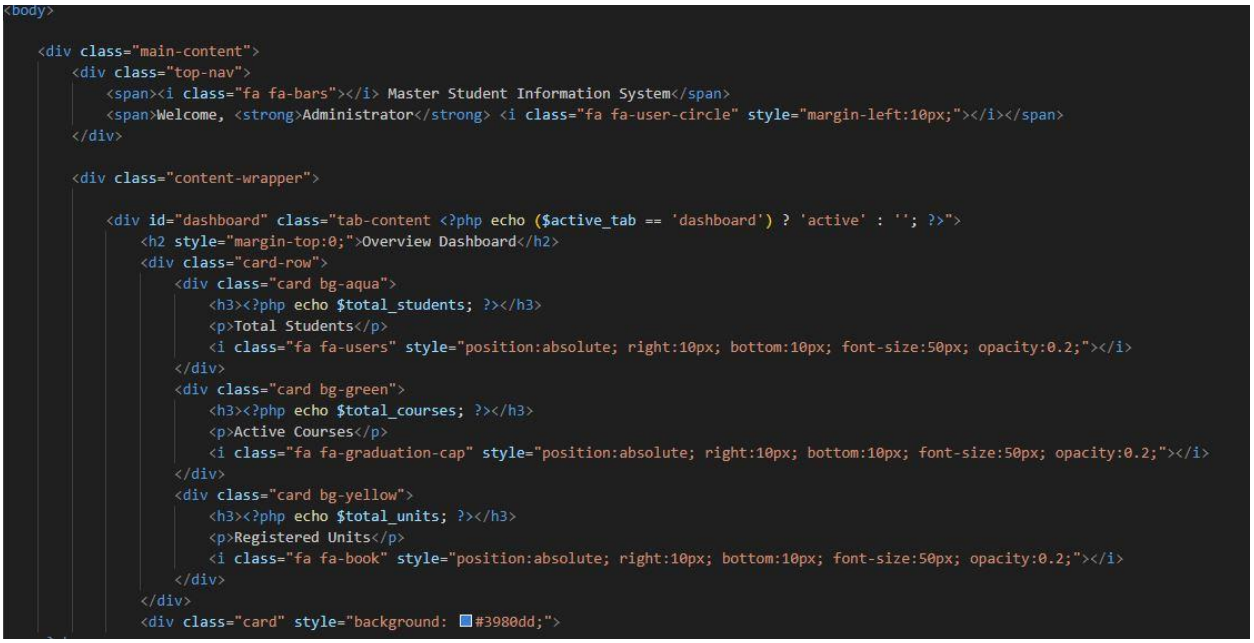
Screenshot of its css

3.2.4 Admin Dashboard Development

This is where the admin view the institute records the fee, units, courses, attendance, marks and the overview of the students registered in the institute.



Admin Page



```
<body>

    <div class="main-content">
        <div class="top-nav">
            <span><i class="fa fa-bars"></i> Master Student Information System</span>
            <span>Welcome, <strong>Administrator</strong> <i class="fa fa-user-circle" style="margin-left:10px;"></i></span>
        </div>

        <div class="content-wrapper">

            <div id="dashboard" class="tab-content <?php echo ($active_tab == 'dashboard') ? 'active' : ''; ?>">
                <h2 style="margin-top:0;">Overview Dashboard</h2>
                <div class="card-row">
                    <div class="card bg-aqua">
                        <h3><?php echo $total_students; ?></h3>
                        <p>Total Students</p>
                        <i class="fa fa-users" style="position:absolute; right:10px; bottom:10px; font-size:50px; opacity:0.2;"></i>
                    </div>
                    <div class="card bg-green">
                        <h3><?php echo $total_courses; ?></h3>
                        <p>Active Courses</p>
                        <i class="fa fa-graduation-cap" style="position:absolute; right:10px; bottom:10px; font-size:50px; opacity:0.2;"></i>
                    </div>
                    <div class="card bg-yellow">
                        <h3><?php echo $total_units; ?></h3>
                        <p>Registered Units</p>
                        <i class="fa fa-book" style="position:absolute; right:10px; bottom:10px; font-size:50px; opacity:0.2;"></i>
                    </div>
                </div>
                <div class="card" style="background: #3980dd;">
```

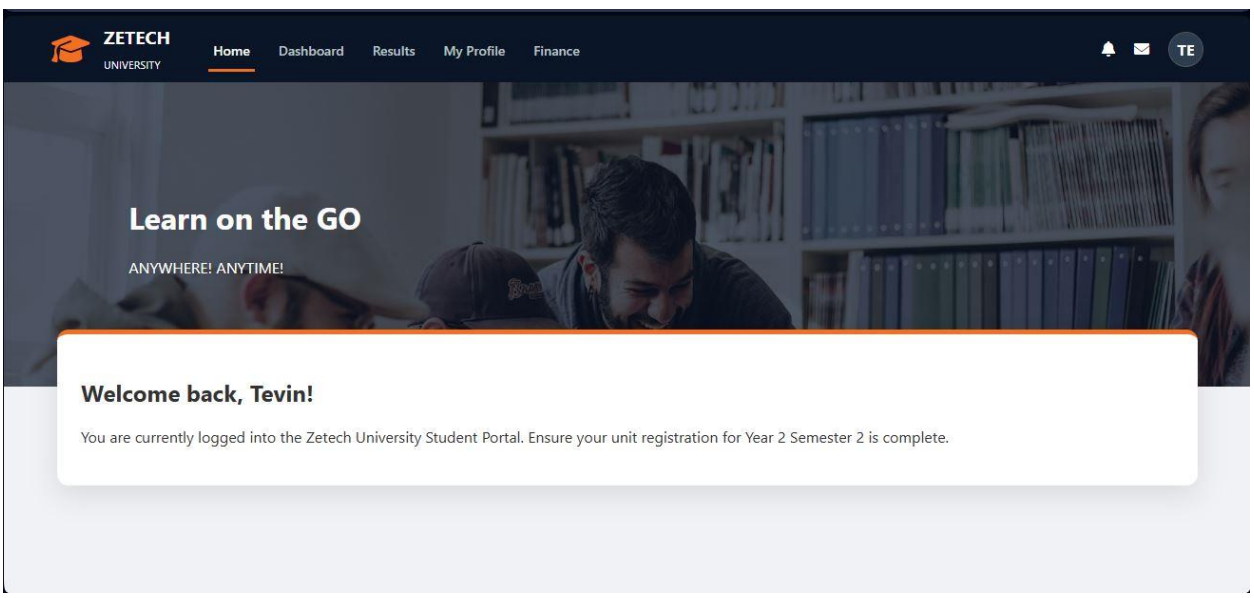
Screenshot of it html

```
<style>
    display: block; border-left: 3px solid transparent; cursor: pointer; transition: 0.3s; }
    .nav-item:hover, .nav-item.active { background: #1e282c; color: #fff; border-left-color: var(--header-green); }
    .nav-item i { margin-right: 10px; width: 20px; }

    .main-content { flex: 1; margin-left: 230px; min-width: 0; }
    .top-nav { background: var(--header-green); padding: 15px 25px;
     color: white; display: flex; justify-content: space-between; align-items: center; box-shadow: 0 2px 4px rgba(0,0,0,0.1); }
    .content-wrapper { padding: 25px; }

    .tab-content { display: none; animation: fadeIn 0.4s; }
    .tab-content.active { display: block; }
    @keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }

    .card-row { display: flex; gap: 20px; margin-bottom: 30px; }
    .card { flex: 1; padding: 20px; border-radius: 4px; color: white; position: relative; overflow: hidden; min-height: 100px; }
    .card h3 { font-size: 38px; margin: 0; position: relative; z-index: 2; }
    .card p { margin: 5px 0 0; font-weight: bold; text-transform: uppercase; font-size: 14px; opacity: 0.9; }
    .bg-aqua { background: #00c0ef; }
    .bg-green { background: #00a65a; }
    .bg-yellow { background: #f39c12; }

    .grid-4 { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; }
    .box { background: #fff; border-top: 3px solid #d2d6de;
     padding: 20px; box-shadow: 0 1px 3px rgba(0,0,0,0.1); margin-bottom: 25px; border-radius: 3px; }
    table { width: 100%; border-collapse: collapse; margin-top: 10px; background: white; }
    th { background: #f9f9f9; padding: 12px 10px; text-align: left; border-bottom: 2px solid #EEE; color: #444; }
    td { padding: 10px; border-bottom: 1px solid #f4f4f4; color: #666; font-size: 14px; }
    tr:hover { background: #fbfbfb; }
```

Screenshot of its css

3.2.5 Student Dashboard Development

The student dashboard is the main interface where students interact with the system. It allows students to view their personal information, registered courses, fee balances and other academic details. The dashboard was developed to provide an easy-to-use and intuitive interface that presents all necessary information in a single view.



```html
</head>
<body>

    <nav class="top-nav">
        <div style="display: flex; align-items: center; gap: 40px;">
            <a href="#" class="nav-brand">
                <i class="fa fa-graduation-cap fa-2x" style="color: var(--accent-orange);"></i>
                <h2 style="margin:0; font-size: 1.2rem;">ZETECH<br><small style="font-weight: normal; font-size: 0.7rem;">UNIVERSITY</small></h2>
            </a>
            <ul class="nav-links">
                <li class="active" id="li-home"><a href="javascript:void(0)" onclick="showSec('home')">Home</a></li>
                <li id="li-dashboard"><a href="javascript:void(0)" onclick="showSec('dashboard')">Dashboard</a></li>
                <li id="li-results"><a href="javascript:void(0)" onclick="showSec('results')">Results</a></li>
                <li id="li-profile"><a href="javascript:void(0)" onclick="showSec('profile')">My Profile</a></li>
                <li id="li-fees"><a href="javascript:void(0)" onclick="showSec('fees')">Finance</a></li>
            </ul>
        </div>

        <div class="user-actions">
    <i class="fa fa-bell"></i>
    <i class="fa fa-envelope"></i>

    <div style="position: relative;">
        <div class="profile-circle" onclick="toggleDropdown()" style="cursor: pointer;">
            <?php echo strtoupper(substr($student['full_name'], 0, 2)); ?>
        </div>

        <div id="profileDropdown" class="dropdown-menu">
            <a href="javascript:void(0)" onclick="showSec('attendance')">
                <i class="fa fa-calendar-check"></i> Attendance
```

Screenshot of its html

```css
/* --- GLOBAL STYLES --- */
:root {
    --primary-color: #0f3460;
    --secondary-color: #e94560;
    --bg-dark: #1a1a2e;
    --sidebar-color: #16213e;
    --text-light: #ffffff;
    --text-dark: #333333;
    --card-shadow: 0 4px 15px rgba(0,0,0,0.1);
}

body {
    margin: 0;
    font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
    background-color: #f4f7f6;
    color: var(--text-dark);
}

/* --- TOP NAVBAR --- */
.top-navbar {
    background-color: var(--primary-color);
    color: white;
    padding: 10px 25px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    position: sticky;
    top: 0;
    z-index: 1000;
    box-shadow: 0 2px 10px rgba(0,0,0,0.2);
```

Screenshot of its css

3.3 Logic Development

3.3.1 login validation code

The login validation process ensures that only authorized users can access the system and when a user enters their username and password in the login form, the system first checks whether the provided details exist in the database. If the credentials match the

stored records, the user is granted access to the system dashboard. If the credentials are incorrect, the system displays an error message.

```php
<?php
session_start();
$conn = new mysqli("localhost", "root", "", "student_db");
if ($conn->connect_error) { die("Connection failed: " . $conn->connect_error); }

$error_msg = "";
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST['login_btn'])) {
    $emailInput    = trim($_POST['email']);
    $usernameInput = trim($_POST['username']);
    $passInput     = trim($_POST['password']);
    $roleInput     = $_POST['role'];

    if ($roleInput == 'Admin') {
        $query = "SELECT * FROM admins WHERE email = ? AND username = ?";
        $redirect_page = "admin.php";
        $stmt = $conn->prepare($query);
        $stmt->bind_param("ss", $emailInput, $usernameInput);
    } else {
        $query = "SELECT * FROM students WHERE email = ? AND full_name = ?";
        $redirect_page = "student_dashboard.php";
        $stmt = $conn->prepare($query);
        $stmt->bind_param("ss", $emailInput, $usernameInput);
    }

    $stmt->execute();
    $result = $stmt->get_result();

    if ($result->num_rows > 0) {
        $user = $result->fetch_assoc();
        if ($passInput === $user['password']) {
            $_SESSION['user_id'] = $user['id'];
```

3.3.2 Forgot password validation code

The forgot password functionality allows users who have forgotten their login credentials to reset their password securely and when a user clicks the “forgot password” option, they are required to enter their registered email address or username. The system then checks whether the provided information exists in the database.

```php
<?php if($msg): ?>
    <div class="msg <?php echo $msg_class; ?>"><?php echo $msg; ?></div>
<?php endif; ?>

<form method="POST" action="">
    <label style="font-size: 12px; font-weight: bold; color: #555;">Username</label>
    <input type="text" name="usernameInput" placeholder="Enter your username" required>

    <label style="font-size: 12px; font-weight: bold; color: #555;">New Password</label>
    <input type="password" name="newPassword" placeholder="••••••••" required>

    <label style="font-size: 12px; font-weight: bold; color: #555;">Confirm Password</label>
    <input type="password" name="confirmPassword" placeholder="••••••••" required>

    <button type="submit">Update Password</button>

    <div class="back-link">
        <a href="login.php">Back to Login</a>
    </div>
</form>
</div>
```

3.3.3 Sign Up Development Validation Code

The sign up functionality allows new users to create an account in the system. During the registration process the system collects user details such as username, email and password. Before storing this information in the database, the system performs validation checks to ensure that all required fields are filled correctly and that the user does not already exist in the system.

```
<h2>Student Sign Up</h2>

<?php if($message!=""){ echo "<p>$message</p>"; } ?>

<form method="POST">

<input type="text" name="full_name" placeholder="Full Name" required>

<input type="text" name="reg_number" placeholder="Registration Number" required>

<input type="email" name="email" placeholder="Email" required>

<input type="password" name="password" placeholder="Password" required>

<input type="text" name="course" placeholder="Course" required>

<select name="year">
<option value="1">Year 1</option>
<option value="2">Year 2</option>
</select>

<select name="semester">
<option value="1">Semester 1</option>
<option value="2">Semester 2</option>
</select>

<button type="submit" name="signup">Sign Up</button>

</form>
```

3.3.4 Admin Dashboard Validation Code

The admin dashboard validation code ensures that only authorized administrators can access the admin section of the system. When a user logs into the system, their role is stored in a session variable and the system then checks this session variable before allowing access to the admin dashboard page.

```
  <h3>KES <?php echo number_format($total_balance, 0); ?></h3>
  <p>Total Fees Owed</p>
  <i class="fa fa-money" style="position:absolute; right:10px; bottom:10px; font-size:50px; opacity:0.2;"></i>
div>
            <div class="box">
                <h4>Quick Statistics</h4>
                <p>The counts above reflect all records currently stored in your SQL database.</p>
            </div>
        </div>

        <div id="students" class="tab-content <?php echo ($active_tab == 'students') ? 'active' : ''; ?>">
            <h2>Student Registry</h2>
            <div class="box">
                <table>
                    <thead>
                        <tr><th>Reg No</th><th>Full Name</th><th>Year/Sem</th><th>Status</th></tr>
                    </thead>
                    <tbody>
                        <?php
                        $st_list = $conn->query("SELECT * FROM students ORDER BY id DESC");
                        if($st_list && $st_list->num_rows > 0):
                            while($s = $st_list->fetch_assoc()): ?>
                                <tr>
                                    <td><strong><?php echo htmlspecialchars($s['reg_number']); ?></strong></td>
                                    <td><?php echo htmlspecialchars($s['full_name']); ?></td>
                                    <td>Year <?php echo $s['year']; ?>, Sem <?php echo $s['semester']; ?></td>
                                    <td><span style="color:green">● Active</span></td>
                                </tr>
                            <?php endwhile;
```

3.3.5 Student Dashboard Validation Code

The student dashboard validation code ensures that only authenticated students can access the student dashboard page. This validation is important because it prevents unauthorized users from accessing student information or performing actions meant for registered students only.

```php
<?php
session_start();

// 1. DATABASE CONNECTION
$conn = new mysqli("localhost", "root", "", "student_db");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// 2. SECURITY CHECK
if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit();
}

$student_id = $_SESSION['user_id'];

// 3. FETCH STUDENT PROFILE
$student_stmt = $conn->prepare("SELECT * FROM students WHERE id = ?");
$student_stmt->bind_param("i", $student_id);
$student_stmt->execute();
$student = $student_stmt->get_result()->fetch_assoc();

// 4. FETCH AUTOMATIC UNITS BASED ON STUDENT YEAR/SEM
$year = $student['year'] ?? 1;
$sem  = $student['semester'] ?? 1;

$units_stmt = $conn->prepare("SELECT * FROM units WHERE year = ? AND semester = ?");
$units_stmt->bind_param("ii", $year, $sem);
$units_stmt->execute();
```

3.3.6 DATABASE DESIGN

The Student Management System uses a MySQL database to store all system-related information. The database stores data for users, students, courses, fees, and academic schedules. Proper relational design ensures data integrity and efficient retrieval.

3.3.6.1 Users Table

Column Name	Data Type	Key/Constraints	Description
id	INT	PK, AI	Unique user identifier
username	VARCHAR	UNIQUE	User login name
email	VARCHAR	UNIQUE	User email
password	VARCHAR		Hashed user password
role	ENUM		User type (Admin or Student)

Table

3.3.6.2 Students Table

Column Name	Data Type	Key/Constraints	Description
student_id	INT	PK, AI	Unique student identifier
name	VARCHAR		Full name of student
course_id	INT	FK → courses.course_id	Linked course

Column Name	Data Type	Key/Constraints	Description
email	VARCHAR	UNIQUE	Student email
phone	VARCHAR		Contact number
registration_date	DATE		Date of student registration

3.3.6.3 Courses Table

Column Name	Data Type	Key/Constraints	Description
course_id	INT	PK, AI	Unique course identifier
course_name	VARCHAR		Name of the course
units	INT		Number of course units

3.3.6.4 Fees Table

Column Name	Data Type	Key/Constraints	Description
fee_id	INT	PK, AI	Unique fee record identifier
student_id	INT	FK → students.student_id	Linked student
amount	DECIMAL		Fee amount
status	ENUM		Paid / Unpaid / Pending

3.4 Testing

System testing was carried out to ensure all the features of the system function correctly and meet the intended requirements the testing involved checking different parts of the system such as user login, registration, dashboard access, database connectivity and data processing. The aim of this process was to identify any errors or bugs in the system and correct them before the final deployment.

Several tests were conducted during the development process to confirm that the system works as expected. Each feature was tested individually to ensure that it performs the correct task and produces the expected results. During testing different user inputs were entered into the system to observe how the system responds and to confirm that validation mechanisms work properly.

The login functionality was tested by entering both correct and incorrect login credentials. When valid credentials were entered, the system successfully granted access to the appropriate dashboard. When incorrect credentials were entered, the system displayed an error message indicating that the login details were invalid. This confirmed that the login validation logic was functioning the right way

The sign-up feature was also tested to ensure that new users could successfully create accounts. During testing the system checked whether duplicate usernames or email addresses existed in the database. If a duplicate record was detected, the system displayed an error message preventing the creation of duplicate accounts. This confirmed that the registration validation mechanism was working properly.

The dashboard access control was also tested to verify that users could only access pages that correspond to their roles. For example, administrators were able to access the admin dashboard while students could only access the student dashboard. Attempts to access unauthorized pages resulted in the system denying access or redirecting the user to the login page.

Database connectivity was tested to ensure that the system could successfully store and retrieve data. The system was able to insert new records, update existing records and display stored information correctly. Any errors encountered during database operations were corrected by reviewing and modifying the SQL queries and database connection settings.

During the testing phase, several issues were identified and corrected for example some pages initially produced warnings due to undefined variables. These issues were resolved by adding proper validation checks to ensure that variables existed before being used. Additional improvements were also made to enhance the system's reliability and user experience.

Overall the testing process helped ensure that all system components work effectively and that the system performs its intended functions accurately and efficiently.

3.5 Deployment

The deployment of the Student Management System was carried out using an online web hosting platform provided by InfinityFree**.** After the system was fully developed and tested locally using the XAMPP server, the project files were uploaded to the hosting server using the file manager provided by the hosting service.

The database required for the system was created using the MySQL database management tool available in the hosting control panel. Necessary tables such as users, students, courses and fee records were created to store the system data. The database connection settings in the system code were then updated to match the hosting server credentials.

After completing the upload and database configuration, the system became accessible through a web browser using the following website link:

http://vesh.infinityfree.me

This allows users such as administrators and students to log in and interact with the system from any device connected to the internet.

The deployment process ensured that the system is fully functional online and can be accessed remotely, making it more practical for real-world use.

CHAPTER FOUR: CONCLUSION AND RECOMMENDATION

4.1 Conclusion

This chapter explains how the system was developed, tested and executed. It describes the logic used in different parts of the system such as login validation, user registration and dashboard access.

Testing was also carried out to ensure that all features of the system work correctly. Errors found during testing were corrected to improve the performance and reliability of the system.

Finally, the system was successfully deployed online using the InfinityFree hosting platform, allowing users to access the system through a web browser.

4.2 Recommendation

Based on the development and testing of the system, several recommendations can be made to improve the system in the future.

First the system can be improved by adding more advanced security features such as password encryption and stronger user authentication to protect user data.

Secondly the system can be enhanced by adding more functionalities such as automated reports, notifications and improved search features to make it more efficient for users.

Third the system could be deployed online so that users can access it from different locations instead of only using it on a local server.

Finally, regular system maintenance and updates should be carried out to ensure that the system continues to function properly and remains secure.

REFERENCES

Welling, L., & Thomson, L. (2017). *PHP and MySQL Web Development* (5th ed.). Addison-Wesley.

Ullman, L. (2018). *PHP for the Web: Visual QuickStart Guide* (5th ed.). Peachpit Press.

W3Schools. (2026). *PHP Form Validation*. Retrieved March 18, 2026, from https://www.w3schools.com/php/php_form_validation.asp